

(Piping | & Multiple Commands)

1 Piping (|)

◆ What piping really is

Piping connects commands through data.

The **output of one command becomes the input of another command** — instantly, without storing anything in a file.

```
command1 | command2
```

Think of it as:

"Take what command1 produces and feed it into command2."

◆ What flows through a pipe

- Only **standard output (stdout)**
- Errors (**stderr**) are **not piped** unless redirected

```
ls | grep ".txt" # works
```

```
ls wrong_dir | grep a # grep gets nothing
```

◆ Why piping is powerful

- Avoids temporary files
- Encourages small, reusable commands
- Enables complex processing with simple tools

🧠 Linux philosophy:

Do one thing well — then pipe it.

◆ Simple examples

```
ls | wc -l
```

👉 Count files in directory

```
ps aux | grep java
```

👉 Find Java processes

```
cat file.txt | sort
```

👉 Sort file content

◆ Multiple pipes (pipeline)

```
ps aux | grep java | awk '{print $2}'
```

Flow:

1. ps aux → list processes
2. grep java → filter Java
3. awk → extract PID column

Each command **transforms data**.

◆ Common pipe-friendly commands

Command	Purpose
grep	filter text
awk	column processing

sed	stream editing
sort	sorting
uniq	remove duplicates
wc	count

◆ Important notes

- Pipes work best with **text output**
- Commands must read from **stdin**
- Order matters

✗ Bad order:

```
wc -l | ls
```

✓ Good order:

```
ls | wc -l
```

2 Multiple Command Execution

Linux allows **running more than one command in a single line** using different operators.

◆ ; — Sequential execution (no conditions)

Definition

Runs commands **one after another, regardless of success or failure.**

```
command1 ; command2 ; command3
```

Behavior

- No dependency
- Every command executes

Example

```
mkdir test ; cd test ; touch file.txt
```

Even if `mkdir test` fails, `cd test` still runs.

🧠 Think of ; as:

"Just run everything."

◆ && — Conditional success execution

Definition

Next command runs **only if the previous command succeeds.**

```
command1 && command2
```

Example

```
make build && echo "Build successful"
```

If `build` fails → `echo` never runs.

🧠 Think of && as:

"Run this **only if** the previous worked."

◆ || — Conditional failure execution

Definition

Next command runs **only if the previous command fails.**

```
command1 || command2
```

Example

```
cd /important || echo "Directory not found"
```

🧠 Think of || as:

"Fallback plan."

◆ Combining operators (real-world style)

```
mkdir logs && cd logs || exit
```

Logic:

- If mkdir succeeds → cd
- If cd fails → exit script